Photonics Data Engineer Intern - Technical Report

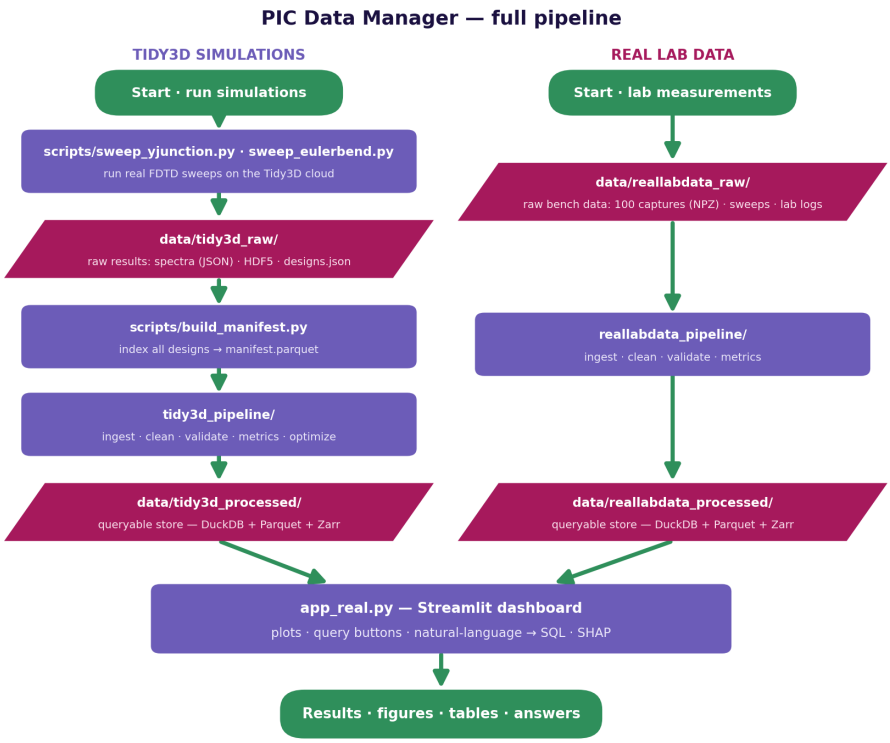
Project	Two pipelines over one storage stack: real Tidy3D FDTD simulations (tidy3d_pipeline, 53 designs) and real cascaded-MZI lab measurements (reallabdata_pipeline, 102 measurements).
Core tools	Python, pandas/NumPy/SciPy, Tidy3D (FDTD), DuckDB + Parquet + Zarr, Pydantic, Streamlit/Plotly, scikit-learn + SHAP, Claude (natural-language SQL).
Demo	streamlit run app_real.py - one command; both stores build automatically on first launch.
Headline results	Best Y-junction 0.155 dB (-0.54 dB via the optimization loop); Euler bend best 0.005 dB. The lab MZM is a two-stage cascade: the multiplicative two-stage fit (R^2 = 0.99997) gives per-stage V_pi 3.20 / 4.60 V; the composite response has -3 dB bandwidth 0.319 MHz and extinction 16.7 dB.
Validation	17 pytest tests + 11 live dashboard checks: known-answer analysis tests, ingest round-trips + idempotency, metadata-vs-manifest validation, raw-file -> stored-metric end-to-end verification, SHA-256 artifact integrity.

1. Architecture

Each dataset has its own pipeline package, but both use the same structure. The data model is a set of Pydantic entities, and each entity gets its own table in DuckDB: Component, SimulationRun, RawArtifact, AnalysisResult and OptimizationRecord for the simulations; Device, Measurement, RawArtifact, AnalysisResult and LabLog for the lab bench. DuckDB holds the metadata and metrics and answers all SQL queries. The measurement payloads live next to it as files: Parquet for data that is tabular - rows x named columns, like a transmission spectrum or a Bode table - and Zarr for data that is N-dimensional arrays, like the 25 x 4096 waveform cubes and the $|E|^2$ field map, where chunking lets one trace be read without loading the whole cube.

Layer	tidy3d_pipeline (simulations)	reallabdata_pipeline (lab bench)
Data model (Pydantic), one DuckDB table each	Component, SimulationRun, RawArtifact, AnalysisResult, OptimizationRecord	Device, Measurement, RawArtifact, AnalysisResult, LabLog
Ingestion adapter	ingest.py - manifest + JSON spectra + HDF5 field maps	ingest.py - NPZ sweeps/captures, device card + lab logs (Markdown)
Store / query	store.py - DuckDB + Parquet + Zarr	store.py - DuckDB + Parquet + Zarr
Analysis	insertion loss, 3 dB bandwidth (computed at ingest for every spectrum)	fringe (incl. two-stage fit), Bode, step features; predictive model - cross-validated RandomForest swing prediction with residual anomaly flags and SHAP attribution
Entry points	cli.py (summary / query / filter) + optimize.py (stretch)	cli.py (metrics / captures / features / logs)

2. Pipeline flow chart



3. Schema

Entity (one DuckDB table each)	Key fields
Component	id, type, name, function, library/source, geometry (swept + fixed parameters), material/platform, expected performance metrics
SimulationRun	run_id, component_id, tool + version, wavelength range, boundary conditions, mesh summary, per-run geometry snapshot, settings, timestamp
RawArtifact	repo-relative path, format, SHA-256 checksum, size
AnalysisResult	metric name / value / units / definition, analysis code version, timestamp
OptimizationRecord	iteration, parameters, objective, best-so-far

Format	Holds	Why
DuckDB	metadata, provenance, metrics - the SQL engine (incl. JSON geometry extraction); can also query the Parquet files in place	one analytical engine for catalog + payloads
Parquet	tabular payloads: transmission spectra, sweep/Bode tables, the design manifest	columnar, compressed, pandas/DuckDB-native
Zarr	N-D arrays: $\|E\|^2$ field map (475x308), 100 waveform cubes (25 traces x 4096 samples)	chunked - slice one trace without reading the cube; attrs carry acquisition metadata

4. Ingestion flow

The adapter is the single seam between a raw result and its stored metrics:

- **Read** - manifest row + JSON spectrum per design (and raw HDF5 when present); the lab adapter reads the NPZ static sweep, FRA, 100 captures, and the Markdown device card / lab logs.

- **Clean** - normalize to a common (wavelength_um, transmission) frame.
- **Validate** - physical sanity checks (monotonic wavelength axis, T in [0, 1.05], no NaNs); bad spectra are rejected, not stored.
- **Store** - provenance to DuckDB; payloads to Parquet/Zarr behind a checksummed RawArtifact with a repo-relative path.
- **Analyze** - metrics computed into analysis_results, no manual step.
- **Query** - runs for a component / compare variants / filter by geometry / retrieve metrics, exposed in the package API, the CLI, and the dashboard (each query button shows its exact SQL, beside a natural-language query box - Claude writes read-only SQL).

5. Design tradeoffs

- **DuckDB + Parquet + Zarr over SQLite** - one analytical engine over catalog and payloads; payloads routed by shape (tables -> Parquet, arrays -> Zarr).
- **Free-form JSON geometry** - new component types need no schema migration; validation lives in Pydantic and in metadata-vs-manifest tests.
- **Content-hashed IDs** - idempotent ingestion over append-only history.
- **RandomForest + SHAP over a neural net** - 22-100 samples per dataset; small-sample-honest cross-validation with exact TreeSHAP attribution.
- **Raw data committed, stores regenerated** - both stores rebuild automatically on first launch; artifact paths are repo-relative so committed DuckDB files resolve on any checkout.

6. Stretch - component optimization

A reproducible gradient-free sweep walks the real Y-junction grid over l_junction with mean insertion loss as the objective; every candidate is stored as an OptimizationRecord (parameters, objective, best-so-far, link to its SimulationRun). Result: l_junction = 2.0 um at 0.155 dB - a 0.54 dB improvement over the baseline. Candidates come from the precomputed FDTD grid (no new solver calls); a finer search would plug a Tidy3D-backed evaluator into the same loop.

7. Assumptions, limitations, and production metadata (simulation integration)

Assumptions. Spectra are power-normalized (loss = -10 log10(T)); single polarization; created_at is ingest time (solver time recoverable via the Tidy3D task_id).

Limitations. Compact sweeps (31/22 designs) on coarse grids; tool_version is not captured on the manifest ingest path; Zarr artifacts are not checksummed yet; complex S-parameters (phase) are not stored, only |T|.

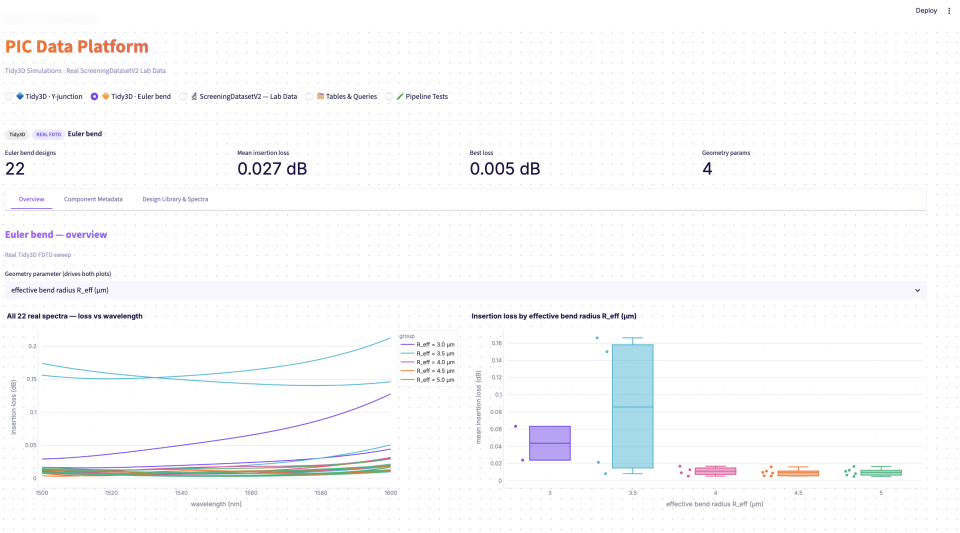
Metadata needed for production. Pinned solver/PDK versions + environment lockfile per run; mesh statistics and convergence flags; solver logs, execution timestamps and cost; a layout/GDS hash linking each run to its exact geometry; a units registry and schema-version column.

8. Next steps

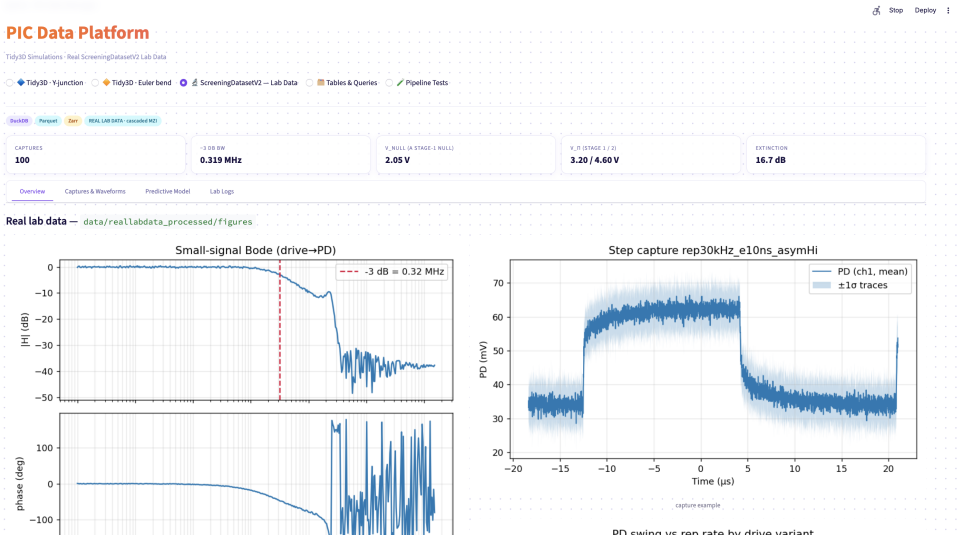
1. **Object storage + CI** - move Parquet/Zarr to S3 (keys already live in RawArtifact.path) and run the test suites in CI with golden-file metric regressions, so the store scales and regressions are caught automatically.
2. **Richer per-run provenance** - pinned tool/PDK versions, mesh/convergence statistics, and solver logs per SimulationRun, closing the biggest current metadata gaps.
3. **Optimization against live FDTD** - swap the precomputed-grid evaluator for a Tidy3D-backed one (Bayesian search, parallel jobs, cost tracking), turning the stretch loop into a real design tool.

Sources: Y-junction adapted from the Tidy3D example library (Y. Zhang et al., Opt. Express 21, 1310-1316 (2013)); Euler bend is a public SOI reference design; FDTD executed on Tidy3D (Flexcompute); lab dataset is the QuEra ScreeningDatasetV2 cascaded-MZI characterization.

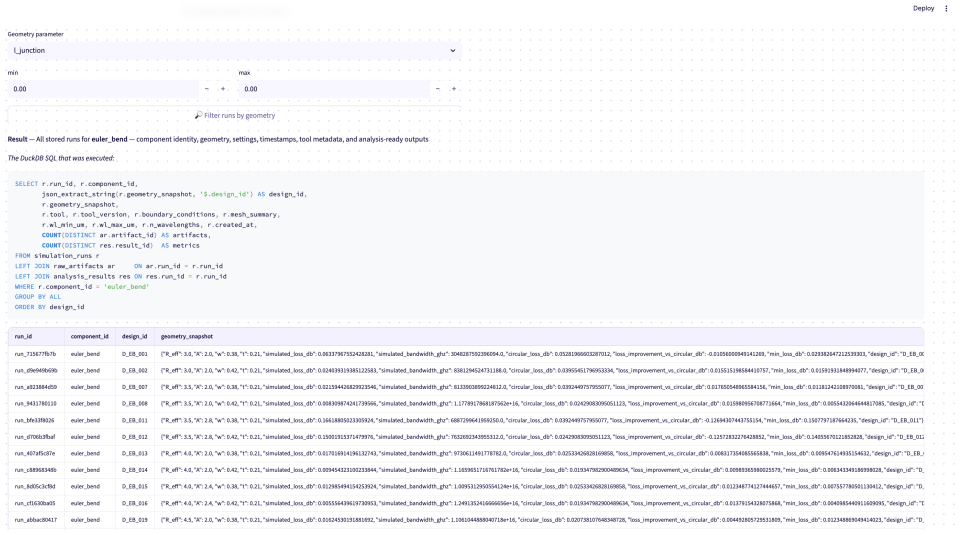
9. Dashboard



Tidy3D component view



Lab-data overview



Tables & Queries console